



# Java Inheritance

AN OVERVIEW OF INHERITANCE IN JAVA

# What is Inheritance?

- ▶ Inheritance is one of the core concepts of object-oriented programming (OOP) in Java. It allows one class (child class) to inherit fields and methods from another class (parent class). This helps in reusing code and establishing a natural hierarchy between classes.
- ▶ Key terms related to Inheritance:
  - ▶ 1. Superclass (Parent Class)
  - ▶ 2. Subclass (Child Class)
  - ▶ 3. 'extends' keyword

# Example 1: Simple Inheritance

- In this example, a subclass inherits attributes and methods from a superclass.

```
class Animal {  
    void eat() { System.out.println('This animal eats food'); }  
}  
  
class Dog extends Animal {  
    void bark() { System.out.println('The dog barks'); }  
}
```

## Example 2: Method Overriding

- ▶ A subclass can override a method defined in its superclass to provide a specific implementation.

```
class Animal {  
    void sound() { System.out.println('Animal makes a  
sound'); }  
}  
class Dog extends Animal {  
    @Override  
    void sound() { System.out.println('Dog barks'); }  
}
```

## Example 3: Using 'super' Keyword

- The 'super' keyword is used to call the constructor or methods of the parent class.

```
class Animal {  
    Animal() { System.out.println('Animal is created'); }  
}  
  
class Dog extends Animal {  
    Dog() { super(); System.out.println('Dog is created'); }  
}
```

# Benefits of Inheritance

- ▶ 1. Code Reusability: Inheritance allows reuse of the code already written in the parent class.
- ▶ 2. Method Overriding: Allows child class to provide specific implementation for a method already defined in parent class.
- ▶ 3. Establishing a Relationship: Creates a parent-child relationship between two classes, allowing hierarchical classification.

# Animal - Inheritance

```
class Animal {  
    private String species;  
  
    public Animal(String species) {  
        this.species = species;  
    }  
  
    public void makeSound() {  
        System.out.println("The " + species + " makes a sound.");  
    }  
  
    public static void main(String[] args) {  
        Animal dog = new Animal("Dog");  
        dog.makeSound();  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Book - Inheritance

```
class Book {  
    private String title;  
    private String author;  
  
    public Book(String title, String author) {  
        this.title = title;  
        this.author = author;  
    }  
  
    public void displayInfo() {  
        System.out.println("Title: " + title);  
        System.out.println("Author: " + author);  
    }  
  
    public static void main(String[] args) {  
        Book myBook = new Book("To Kill a Mockingbird", "Harper Lee");  
        myBook.displayInfo();  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Car - Inheritance

```
class Car {  
    private String make;  
    private String model;  
  
    public Car(String make, String model) {  
        this.make = make;  
        this.model = model;  
    }  
  
    public void start() {  
        System.out.println("The " + make + " " + model + " is starting.");  
    }  
  
    public void drive() {  
        System.out.println("The " + make + " " + model + " is driving.");  
    }  
  
    public static void main(String[] args) {  
        Car myCar = new Car("Toyota", "Camry");  
        myCar.start();  
        myCar.drive();  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Person - Inheritance

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public void introduce() {  
        System.out.println("Hi, I'm " + name + " and I'm " + age + " years old.");  
    }  
  
    public static void main(String[] args) {  
        Person person = new Person("Alice", 30);  
        person.introduce();  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Plant - Inheritance

```
class Plant {  
    private String name;  
  
    public Plant(String name) {  
        this.name = name;  
    }  
  
    public void grow() {  
        System.out.println("The " + name + " is growing.");  
    }  
  
    public static void main(String[] args) {  
        Plant flower = new Plant("Rose");  
        flower.grow();  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

**Reset Student**

**Leave Student View**

# Example 1: Handling ArithmeticException (Full Code)

```
public class ArithmeticExceptionExample {  
    public static void main(String[] args) {  
        try {  
            int result = divide(10, 0);  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("An arithmetic exception occurred: " + e.getMessage());  
        }  
    }  
  
    public static int divide(int dividend, int divisor) {  
        return dividend / divisor;  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Example 2: Custom Exception (Full Code)

```
class CustomException extends Exception {  
    public CustomException(String message) {  
        super(message);  
    }  
}  
  
public class CustomExceptionExample {  
    public static void main(String[] args) {  
        try {  
            validateAge(15);  
        } catch (CustomException e) {  
            System.out.println("Custom exception occurred: " + e.getMessage());  
        }  
    }  
  
    public static void validateAge(int age) throws CustomException {  
        if (age < 18) {  
            throw new CustomException("Age must be at least 18.");  
        } else {  
            System.out.println("Age is valid.");  
        }  
    }  
}
```

6d You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)

# Example 3: Multiple Exceptions (Full Code)

```
public class MultipleExceptionsExample {  
    public static void main(String[] args) {  
        try {  
            int result = divide(10, 0);  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic exception occurred: " + e.getMessage());  
        } catch (NullPointerException e) {  
            System.out.println("Null pointer exception occurred: " + e.getMessage());  
        }  
    }  
  
    public static int divide(int dividend, int divisor) {  
        if (divisor == 0) {  
            throw new ArithmeticException("Division by zero");  
        }  
        return dividend / divisor;  
    }  
}
```

63 You are currently logged into  
Student View

*Resetting the test student will clear all history for  
this student, allowing you to view the course as a  
brand new student.*

[Reset Student](#)

[Leave Student View](#)